ПРАКТИЧЕСКИЙ УЧЕБНИК • 2026

XPolicyLab

Максимально понятный гайд для человека без подготовки

От камеры и текстовой команды — до action chunk, WebSocket и движения робота.
С ясной маркировкой: что относится к VLA, что — ко всем policy, что — к XPolicyLab, а что — к симулятору и железу.

Основано на публичном репозитории XPolicyLab
commit 092782d22702b1600ed3ec3d17b48418b2e42a77 • 4 сентября 2026

Как читать этот гайд

Этот текст построен слоями. Сначала — одна простая картинка в голове. Потом — точные термины. Затем — устройство репозитория и практические команды. Если код пока пугает, читайте главы 1–6 и шпаргалку; к практике возвращайтесь позже.

ВАЖНО

Робот может причинить физический ущерб. Любую новую policy сначала проверяйте в debug-режиме и симуляторе, затем на малой скорости, с ограничениями действий, зоной безопасности и доступной аварийной остановкой.

Легенда маркировки

Метка	Как понимать
ВСЕ POLICY	Верно для VLA, ACT, Diffusion Policy, planner и других способов выбора действий.
ТОЛЬКО / ОСОБЕННО VLA	Текст, языковые токены, VLM-компоненты и мультимодальная предобработка.
ИНФРАСТРУКТУРА XPOLICYLAB	Адаптеры, сервер, клиент, сеть, lifecycle и форматы.
СИМУЛЯТОР / РОБОТ	Физика, ROS, драйверы, камеры, моторы. Это не часть VLA.

Оглавление

Как читать этот гайд 2

Легенда маркировки . . . 2

Оглавление . . . 3

1. Самая короткая версия 6

Почему это вообще нужно . . . 6

2. Что такое policy — с самого нуля 7

Policy — правило выбора следующего действия . . . 7

Policy не обязательно нейросеть . . . 7

Observation: что робот знает сейчас . . . 7

Action: какой именно язык команд . . . 7

3. VLA, imitation learning, diffusion и world-action models 8

VLA: Vision + Language + Action . . . 8

Imitation learning . . . 8

ACT: Action Chunking with Transformers . . . 8

Diffusion Policy . . . 9

World model и world-action model . . . 9

4. Пять примеров моделей в XPolicyLab 10

π0 и π0.5 . . . 10

OpenVLA и OpenVLA-OFT . . . 10

GR00T . . . 10

ACT и Diffusion Policy . . . 10

5. Ментальная модель архитектуры — каждый блок 11

Левый столбец: heterogeneous policies . . . 11

Центр: unified XPolicyLab infrastructure . . . 11

Правый столбец: evaluation environments . . . 12

6. Что происходит «по проводам» 12

Один эпизод по шагам . . . 12

WebSocket, RPC и MessagePack — три разные вещи . . . 12

Надёжность: retry, keepalive, exactly-once, restart detection . . . 13

Тонкости сериализации . . . 13

7. Observation, RGB-D и proprioception 14

Стандартный observation XPolicyLab . . . 14

RGB и BGR . . . 14

Depth и геометрия камеры . . . 14

Proprioception . . . 14
Поза end-effector . . . 14

8. Action chunks, stateful policy и batch inference 15

Зачем action chunk . . . 15
Stateful policy . . . 15
Batch inference . . . 15
Частота управления . . . 15

9. Adapter, model.py и путь через сеть 16

Adapter — переводчик, не новая модель . . . 16
Обязательные и дополнительные методы . . . 16
Простейший skeleton . . . 16

10. Runtime, checkpoint, config и вычислительный стек 17

Код, веса и настройки — не одно и то же . . . 17
PyTorch и JAX . . . 17
CUDA и GPU . . . 17
Зачем dependency isolation . . . 17
GPU separation . . . 18

11. Что где живёт в репозитории 18

client_server/ws . . . 18
policy/ . . . 19
Базовые зависимости пакета . . . 19

12. Lifecycle: install → data → train → serve → evaluate 20

Data processing . . . 20
Training . . . 20
Inference . . . 20
Evaluation . . . 20
Checkpoint naming . . . 20

13. Что нужно установить 21

Минимум для изучения и debug . . . 21
Для настоящей VLA . . . 21
Для симулятора/робота . . . 21
Проверочный список совместимости . . . 21

14. Как запустить demo_policy 22

Шаг 1. Клонировать и поставить базовый пакет . . . 22
Шаг 2. Установить demo adapter . . . 22
Шаг 3. Offline debug без simulator . . . 22

Что именно проверяется . . . 22
Дополнительная проверка encoded images . . . 22

15. Как подключить свою модель 23

Практический маршрут . . . 23
Решения, которые обязан принять adapter author . . . 23
Проверки из репозитория . . . 23
Минимальный критерий качества . . . 23

16. Как подключить свой simulator или robot 24

Environment client делает четыре вещи . . . 24
Граница ответственности . . . 24
Same-machine и split-machine . . . 24
Безопасность реального робота . . . 24

17. Что XPolicyLab НЕ делает 25

Главное ограничение унификации . . . 25

18. Диагностика: где искать проблему 26

Где смотреть traceback . . . 26
Порядок изоляции причины . . . 26

19. FAQ 27

20. Шпаргалка на одну страницу 28

Пять функций, которые надо помнить . . . 28
Самый безопасный путь запуска . . . 28
Красные флаги . . . 28

21. Полный словарь терминов 29

22. Источники и границы актуальности 31

Основной источник . . . 31
Модели и методы . . . 31
Benchmarks . . . 31
Ограничения этого учебника . . . 31

1. Самая короткая версия

XPolicyLab — не мозг робота и не симулятор. Это стандартизирующий слой между моделью, которая выбирает действия, и средой, которая даёт наблюдения и исполняет команды.

Один цикл принятия решения



Дальше environment исполняет действия и снимает новое observation

ВСЕ POLICY

Любую policy можно представить как функцию: **observation → action**. У неё могут быть память, планировщик, нейросеть или набор правил — формула остаётся полезной.

ТОЛЬКО / ОСОБЕННО VLA

У VLA observation обычно включает изображение и текстовую инструкцию. Слово **Action** в VLA означает, что выход связан с управлением роботом, а не просто является текстовым ответом.

ИНФРАСТРУКТУРА XPOLICYLAB

XPolicyLab заставляет разные модели выглядеть одинаково снаружи: **update_obs → get_action → reset**, плюс batch-методы. Затем вызывает их через policy server.

СИМУЛЯТОР / РОБОТ

Среда создаёт изображение и состояния суставов, исполняет action и считает успех. Она может быть симулятором или реальным роботом; сама по себе она не является VLA.

Почему это вообще нужно

Без общего слоя каждая модель интегрируется с каждой средой отдельно. Для N моделей и M сред получается до N×M связок. XPolicyLab предлагает один адаптер на модель и один клиент на среду: порядка N+M интеграций. Это инженерное упрощение, а не обещание, что любая модель автоматически заработает на любом роботе.

```
БЕЗ СТАНДАРТА                  С XPOLICYLAB
Pi0 → RoboDojo                   Pi0 ─┐
Pi0 → RoboTwin                OpenVLA ├→ единый контракт → RoboDojo
ACT → RoboDojo                   ACT ┤                   → RoboTwin
ACT → RoboTwin                    DP ┘                   → Robot
```

2. Что такое policy — с самого нуля

Policy — правило выбора следующего действия

Представьте водителя. Он видит дорогу, скорость и маршрут — это observation. Он поворачивает руль и нажимает педали — это action. Policy — всё, что принимает первое и решает второе.

```
observation_t = {камера, состояние суставов, команда}
action_t:t+H = policy(observation_t, внутренняя_память)
```

Индекс **t** означает текущий момент. **H** — горизонт: сколько будущих шагов возвращает модель. Если H=1, это одно действие; если H=16, это action chunk из шестнадцати.

Policy не обязательно нейросеть

- Жёстко заданное правило: «если датчик близко — остановиться».
- Классический planner: строит путь по геометрии и ограничениям.
- Imitation policy: учится на действиях человека/эксперта.
- VLA: объединяет зрение, язык и действия.

В model_template.py прямо указано, что self.model может быть нейросетью, planner-ом или любым policy-объектом. Значит, XPolicyLab шире, чем VLA.

Observation: что робот знает сейчас

Observation — не объективная реальность, а доступный системе пакет данных. Если камера не видит предмет за спиной, policy тоже его не видит — если только не хранит память о прошлом.

Часть observation	Пример	Кто создаёт
Vision	RGB с head-камеры и wrist-камер	Камеры / симулятор
Depth	Расстояние до поверхности для каждого пикселя	Depth-камера / симулятор
State	Углы суставов, поза захвата, состояние базы	Контроллер / симулятор
Instruction	«Положи красную чашку в мойку»	Пользователь / benchmark
Metadata	env_idx, частота, идентификаторы испытания	Обвязка среды

Action: какой именно язык команд

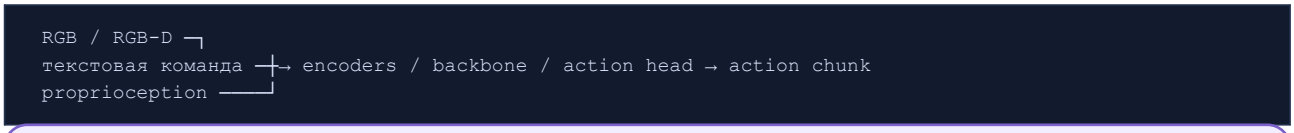
Слово action ничего не гарантирует без action space. Вектор из семи чисел может означать целевые углы, их изменения, скорости или позу инструмента. Даже правильные числа в неправильной интерпретации опасны.

Пространство	Смысл	Сильная сторона	Риск
Joint	Команды каждому суставу	Однозначно для конкретного робота	Плохо переносится между роботами
EE pose	Куда переместить рабочий конец	Интуитивно и ближе к задаче	Нужна inverse kinematics/контроллер
Delta EE	Небольшое изменение позы	Локально устойчиво	Ошибки могут накапливаться
Velocity/torque	Скорость или усилие	Точное динамическое управление	Требует высокой частоты и строгой безопасности

3. VLA, imitation learning, diffusion и world-action models

VLA: Vision + Language + Action

VLA — класс мультимодальных моделей. **Vision** сообщает, что видно. **Language** задаёт цель или контекст. **Action** связывает решение с телом робота. Упрощённо:



ТОЛЬКО / ОСОБЕННО VLA

Tokenizer обычно нужен для текста: разбивает фразу на токены и превращает их в числа. Processor собирает изображения, текст и state в тот формат, который ждёт конкретная модель. Это внутренности VLA/адаптера, а не задача environment client.

Чем VLA отличается от обычной policy

Вопрос	Обычная узкая policy	VLA
Вход	Часто state и/или фиксированные камеры	Обычно vision + language + state
Задача	Одна или несколько заранее заданных	Стремится понимать много формулировок и ситуаций
Предобучение	Может обучаться только на robot demos	Часто использует знания vision-language pretraining
Выход	Action или chunk	Тоже action или chunk — магии на проводе нет
XPolicyLab API	update_obs/get_action/reset	Тот же API

Imitation learning

Эксперт показывает демонстрации. В каждой точке есть observation и желаемое action. Модель подбирает веса так, чтобы воспроизводить действия эксперта. Это похоже на обучение вождению по записям опытного водителя — без самостоятельного исследования всех аварийных вариантов.

ВСЕ POLICY

VLA тоже может обучаться imitation learning. «VLA» описывает тип входов/модели, а «imitation learning» — способ обучения. Эти категории не взаимоисключающие.

ACT: Action Chunking with Transformers

ACT — imitation policy, известная тем, что предсказывает последовательность действий и использует transformer-архитектуру. Chunk уменьшает необходимость заново спрашивать модель на каждом микрошаге и помогает задавать согласованное движение. В XPolicyLab ACT находится в policy/ACT/, upstream-код включает DETR-компоненты; важные настройки — chunk_size, temporal_agg, размеры скрытых слоёв и путь к checkpoint.

Diffusion Policy

Diffusion Policy генерирует траекторию итеративно. Сначала имеется шумоподобный кандидат действий, затем сеть много раз «очищает» его с учётом observation. Аналогия: не написать маршрут одним движением, а несколько раз уточнить черновик. Это не текстовый генератор и не обязательно VLA.

ВСЕ POLICY

Слово diffusion описывает механизм генерации. Policy может быть visuomotor без языка; добавление текста возможно, но не является обязательным свойством Diffusion Policy.

World model и world-action model

World model учится предсказывать состояние или наблюдение будущего: «что случится, если…». Policy отвечает: «что сделать». World-action models пытаются тесно связать представление динамики мира и выбор действий. В 2026 году термин WAM используется разными работами не совершенно одинаково, поэтому всегда проверяйте определение конкретного paper. XPolicyLab не навязывает единую внутреннюю архитектуру WAM — он стандартизирует границу ввода/вывода.

4. Пять примеров моделей в XPolicyLab

Пример	Класс	Главная идея для новичка	В XPolicyLab
π0 / π0.5	VLA + flow/action expert	Общая модель управления; π0.5 усиливает open-world generalization через разнородное co-training	uv/OpenPI, JAX-экосистема, LeRobot data
OpenVLA-OFT	VLA	Vision-language backbone превращает мультимодальный контекст в действия; OFT улучшает fine-tuning/action decoding	7B base, TFDS/ALOHA format, отдельный conda runtime
GR00T-N1.7	Foundation VLA для роботов	VLM кодирует vision/text, action-модуль использует state и действия	uv runtime, LeRobot datasets, NVIDIA checkpoints
ACT	Imitation policy	Transformer выдаёт action chunks из демонстраций	Conda, HDF5/конвертация, chunk_size
Diffusion Policy	Imitation / generative policy	Денойзит последовательность действий	Conda, Zarr dataset, checkpoint

π0 и π0.5

π0 описана как Vision-Language-Action Flow Model. Практически для пользователя XPolicyLab важнее не формулы flow matching, а граница интеграции: OpenPI runtime загружает config, нормализационную статистику и checkpoint, processor преобразует XPolicyLab observation, а модель выдаёт chunk. Адаптеры находятся в policy/Pi_0/ и policy/Pi_05/.

π0.5 основана на π0 и использует co-training на разнородных источниках: данные разных роботов, web data и семантические задачи. Это относится к способности самой VLA обобщать, а не к сетевому мосту XPolicyLab.

OpenVLA и OpenVLA-OFT

Оригинальная OpenVLA — открытая 7B VLA, обученная на большом наборе реальных robot demonstrations. Репозиторий XPolicyLab интегрирует вариант OpenVLA-OFT: базовые веса openvla-7b, данные ALOHA-format TFDS, fine-tuned checkpoint и параметры вроде use_proprio, use_diffusion, num_images_in_input. Это хороший пример того, как много model-specific деталей прячется за одинаковым get_action().

GR00T

GR00T — семейство NVIDIA для generalist robot foundation models. Адаптер в актуальном репозитории называется GR00T_N17 и по умолчанию ссылается на nvidia/GR00T-N1.7-3B и отдельную Cosmos-модель. Не путайте: XPolicyLab не разрабатывает GR00T и не превращает другую policy в GR00T; он только оборачивает конкретную реализацию.

ACT и Diffusion Policy

Обе часто выступают как сильные imitation-learning baselines. ACT строит chunk трансформером, Diffusion Policy — итеративной диффузионной генерацией. Ни одной не нужен язык по определению. Поэтому они помогают увидеть, что XPolicyLab — стандарт для robot policies вообще, а не только каталог VLA.

ВАЖНО

Названия моделей и версии быстро меняются. Для установки и конкретного checkpoint всегда считайте README внутри policy/<POLICY>/ источником истины. Этот гайд объясняет архитектуру и проверенное состояние репозитория на указанную дату.

5. Ментальная модель архитектуры — каждый блок

Ниже — официальная схема assets/infra.png из актуального репозитория. Она совпадает по содержанию с описанной пользователем ментальной моделью: heterogeneous policies слева, unified infrastructure в центре и environments справа.

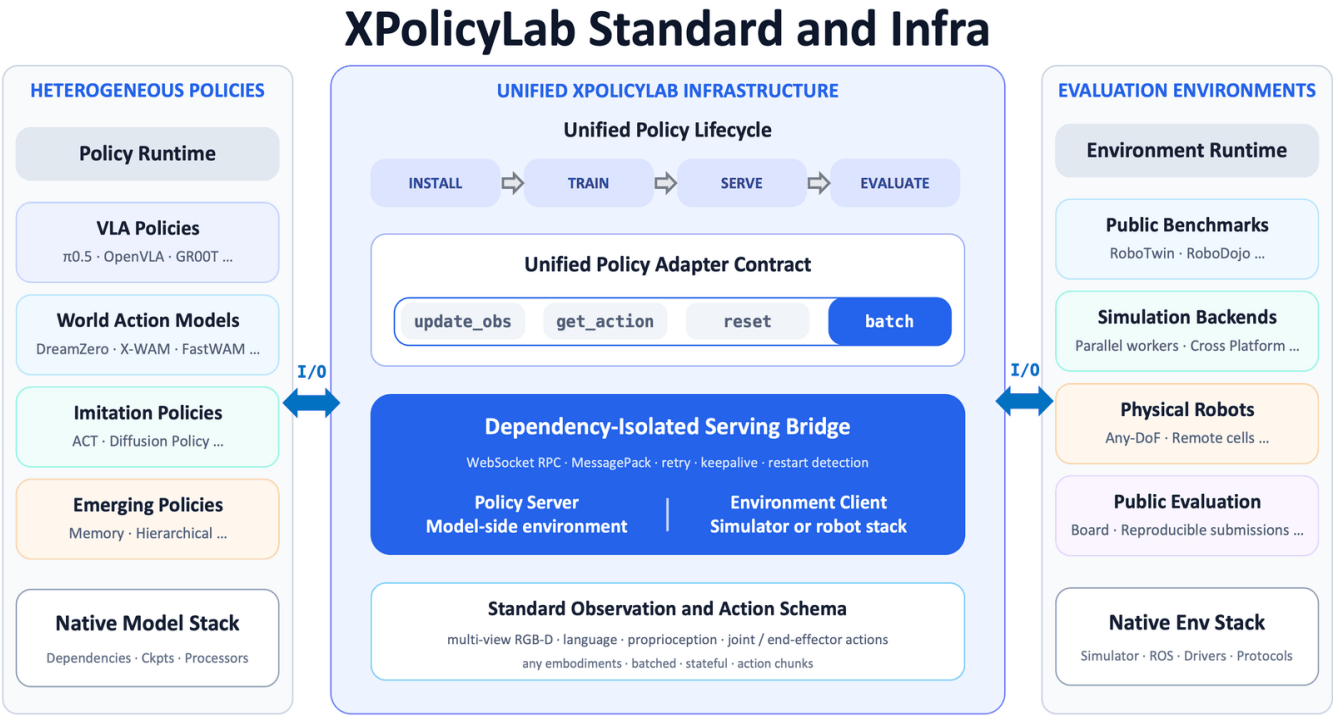


Рисунок 1. Официальная архитектурная схема XPolicyLab. Источник: репозиторий XPolicyLab.

Левый столбец: heterogeneous policies

- **VLA policies** — π0.5, OpenVLA, GR00T и другие модели, использующие vision-language-action.
- **World Action Models** — отдельное семейство подходов, связывающее динамику/будущее и действия.
- **Imitation policies** — ACT и Diffusion Policy как примеры обучения по демонстрациям.
- **Emerging policies** — memory-augmented, hierarchical и другие новые архитектуры.
- **Native model stack** — родные зависимости, checkpoints и processors. XPolicyLab не заставляет их стать одинаковыми внутри.

Центр: unified XPolicyLab infrastructure

- **Lifecycle install → train → serve → evaluate** — одинаковая форма верхнеуровневых скриптов, но не идентичные внутренности обучения.
- **Adapter contract** — update_obs, get_action, reset и batch-варианты.

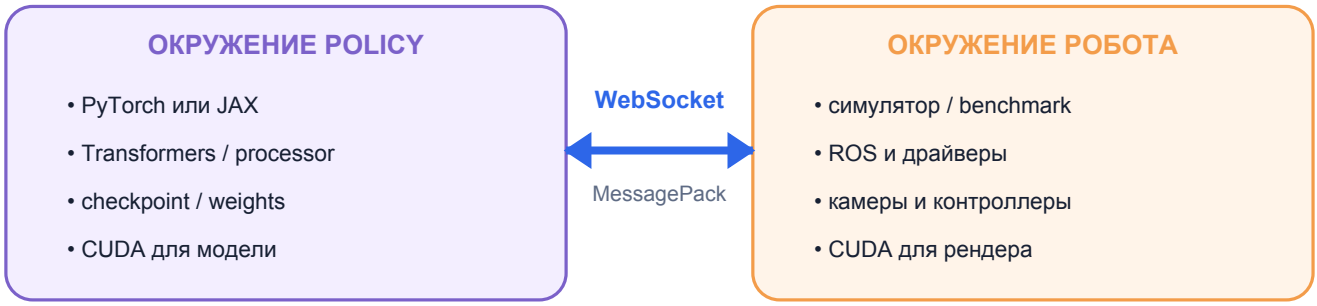
- **Dependency-isolated serving bridge** — модель и среда живут отдельно и общаются по сети.
- **Standard observation/action schema** — общие ключи для камер, state и actions.

Правый столбец: evaluation environments

- **Public benchmarks** — RoboDojo и RoboTwin задают задачи, метрики и данные.
- **Simulation backends** — физика, рендер, параллельные workers. Это не VLA.
- **Physical robots** — реальное железо с любым числом DoF, если написан совместимый client/adapter.
- **Native env stack** — simulator, ROS, drivers, protocols. Они остаются на стороне среды.

6. Что происходит «по проводам»

Разные процессы, разные зависимости, при желании разные компьютеры



Один эпизод по шагам

Шаг	Что происходит	Где
0	Policy server загружает config, processor и checkpoint; затем открывает порт	Policy runtime
1	Environment client подключается и делает HELLO handshake	WebSocket
2	Перед новым trial вызывается reset(); счётчик inference начинается заново	Model adapter
3	Среда собирает камеры, proprioception и instruction	Simulator / robot
4	Observation сериализуется в MessagePack; numpy arrays сохраняют dtype/shape	Client / codec
5	Сервер декодирует цветные камеры и вызывает update_obs(obs)	Policy server
6	get_action() запускает preprocessing → network → postprocessing	Adapter/model
7	Action chunk сериализуется и возвращается клиенту	WebSocket
8	Environment проверяет форму/ключи, исполняет один или несколько шагов	Environment
9	Цикл повторяется до успеха, таймаута или ошибки; trial_end сообщает результат	Обе стороны

WebSocket, RPC и MessagePack — три разные вещи

WebSocket — дорога с постоянным двусторонним соединением. **RPC** — договорённость, что сообщение означает вызов функции. **MessagePack** — упаковочная коробка для данных. Путать их — всё равно что называть грузовик, услугу доставки и картонную коробку одним словом.

```
Python dict / numpy arrays
        ↓ encode_frame (MessagePack + NumPy support)
WebSocket binary frame
        ↓ сеть
decode_envelope → проверка Frame → вызов Model метода
        ↓
result → MessagePack → WebSocket → environment client
```

Надёжность: retry, keepalive, exactly-once, restart detection

- Каждый запрос имеет request_id. При reconnect клиент повторяет тот же id, а сервер может вернуть кэшированный ответ вместо повторного движения stateful policy.
- Если старый запрос ещё выполняется, дубликат ожидает тот же task. Это защищает от двойного неидемпотентного вызова.
- Timeout считается опасным: сервер мог продолжить вычисление, поэтому trial нужно завершить как ошибочный, а не бездумно повторять.
- HELLO_ACK содержит instance id сервера. Если после reconnect id другой, сервер перезапускался и потерял состояние модели; клиент поднимает ошибку.
- WebSocket ping/pong работает как keepalive. В клиенте event loop вынесен в отдельный поток, чтобы долгий шаг симулятора не мешал отвечать на ping.

Тонкости сериализации

- torch.Tensor преобразуется в NumPy без обязательного импорта torch на стороне environment.
- CUDA tensor переносится на CPU; bfloat16 расширяется до float, потому что NumPy не имеет прямого bfloat16.
- Tuple после передачи становится list; integer keys нормализуются в строки; декодированные массивы могут быть read-only views.
- Object-dtype NumPy arrays запрещены — они плохо определены для безопасной межпроцессной передачи.

7. Observation, RGB-D и proprioception

Стандартный observation XPolicyLab

```
obs = {
  'instruction': 'поставь чашку на поднос',
  'vision': {
    'cam_head': {'color': RGB[H,W,3], 'depth': Depth[H,W]},
    'cam_left_wrist': {'color': RGB[H,W,3]},
  },
  'state': {
    'left_arm_joint_state': float[DOF],
    'left_ee_joint_state': float[EEF_DOF],
    'left_ee_pose': [x,y,z,qw,qx,qy,qz]
  }
}
```

Поля optional: конкретная среда может не иметь depth или второй wrist-камеры. Но конкретная policy может требовать их. Общая схема данных не означает автоматическую совместимость — адаптер обязан валидировать вход.

RGB и BGR

XPolicyLab декларирует RGB end-to-end. Сервер декодирует runtime camera colors до вызова update_obs; model.py не должен декодировать изображение повторно. Для сохранённых trajectory bits используется только decode_image_bit, который сглаживает различия старых форматов и возвращает RGB.

ВАЖНО

Если случайно поменять RGB↔BGR, красный объект станет для модели синим. Размер массива останется правильным, поэтому ошибка может выглядеть как «плохая модель», а не как баг данных.

Depth и геометрия камеры

Depth хранит расстояние, а intrinsic matrix описывает геометрию самой камеры. Extrinsics связывают координаты камеры с общей системой координат. Они относятся к сенсорам/среде; VLA может использовать их, игнорировать или получать только RGB.

Proprioception

Это внутреннее чувство положения тела. Для руки — углы суставов и состояние gripper; для мобильной базы — pose и twist. Без proprioception изображение может показывать сцену, но не сообщать модели точное положение скрытого сустава.

Поза end-effector

В XPolicyLab позы имеют порядок [x, y, z, qw, qx, qy, qz]. Первые три числа — позиция. Последние четыре — quaternion ориентации. Порядок компонентов критичен: библиотеки иногда используют [qx,qy,qz,qw]. Адаптер обязан перевести правильно.

8. Action chunks, stateful policy и batch inference

Зачем action chunk

GPU-inference может занимать десятки или сотни миллисекунд, а робот хочет команды чаще. Policy поэтому может вернуть сразу H будущих действий. Environment исполняет весь chunk или его часть, затем снова спрашивает модель.

Плюс	Цена
Меньше сетевых round trips и inference-вызовов	Модель дольше действует без новой информации
Движение может быть временно согласованнее	Старый chunk устаревает, если мир внезапно изменился
GPU используется эффективнее	Нужно решить, сколько действий исполнять и как перекрывать chunks

Stateful policy

Stateful policy хранит историю изображений, hidden state, текущий план или неиспользованный хвост chunk. Поэтому reset() — семантически важная граница эпизода. Если забыть reset, память предыдущей задачи протечёт в следующую.

Batch inference

При benchmark evaluation можно запустить несколько environment workers. update_obs_batch(obs_list) принимает список наблюдений, а get_action_batch(env_idx_list) возвращает chunks, выровненные по активным индексам сред. Это увеличивает загрузку GPU, но усложняет управление state для каждой среды.

ИНФРАСТРУКТУРА XPOLICYLAB

В текущем WebSocket-клиенте счётчик step считает именно inference-вызовы get_action/get_action_batch, а не низкоуровневые control steps. Один inference может вернуть целый chunk.

Частота управления

Если среда работает на 20 Hz, один control step длится 50 ms. Chunk из 10 шагов покрывает примерно 0,5 s. Но только если action semantics, interpolation и реальная частота совпадают. Поле frequency и timing нужно считать частью контракта эксперимента.

9. Adapter, model.py и путь через сеть

Adapter — переводчик, не новая модель

```
общий XPolicyLab obs
        ↓  model.py: выбрать камеры, resize, normalize, tokenize
родной input модели
        ↓  self.model / network
родной output модели
        ↓  denormalize, clip, переименовать keys
общий XPolicyLab action chunk
```

Обязательные и дополнительные методы

Метод	Что обязан сделать	Типичная ошибка
__init__(model_cfg)	Прочитать config, загрузить processor/model/checkpoint	Загрузить не те weights или не то device
update_obs(obs)	Сохранить/предобработать одно observation	Повторно декодировать RGB
get_action()	Вернуть list[action_dict] — chunk	Неверные keys, shape, dtype, units
update_obs_batch	Обновить state нескольких env	Перепутать env order
get_action_batch	Вернуть batch chunks по активным env	Потерять state неактивных env
reset()	Очистить эпизодическую память; без аргументов	Оставить history/chunk предыдущего trial
prepare_case	Опционально подготовить новый action case	Считать обязательным для всех моделей
on_trial_end	Опционально получить итог trial	Использовать как скрытый training feedback

Простейший skeleton

```python
class Model(ModelTemplate):
    def __init__(self, model_cfg):
        self.processor = load_processor(model_cfg)
        self.model = load_network_and_weights(model_cfg)
        self.obs = None

    def update_obs(self, obs):
        self.obs = obs                # color уже RGB array

    def get_action(self):
        inputs = self.processor(self.obs)
        native = self.model.predict(inputs)
        return convert_to_action_chunk(native)

    def reset(self):
        self.obs = None
        self.model.reset_state_if_needed()
```

Этот код концептуальный: реальные методы загрузки и predict различаются. Главное — маленькая и явная граница перевода.

10. Runtime, checkpoint, config и вычислительный стек

Код, веса и настройки — не одно и то же

Объект	Аналогия с человеком	Что случится, если отсутствует
Model code	Строение мозга и алгоритм мышления	Нечего исполнять
Weights	Выученная память/навык	Сеть случайна или базовая
Checkpoint	Снимок обучения	Нельзя воспроизвести нужный run
Config	Инструкция, какой мозг и режим собрать	Shape/path/режим могут не совпасть
Processor/tokenizer	Органы чувств и переводчик	Модель получает не тот масштаб/токены
Runtime	Комната, инструменты и электричество	Код не импортируется или не видит GPU

PyTorch и JAX

Оба работают с тензорами, автоматическим дифференцированием и GPU. Но у них разные API, механизмы компиляции, форматы состояния и экосистемы. Нельзя считать checkpoint PyTorch автоматически совместимым с JAX даже при похожей архитектуре.

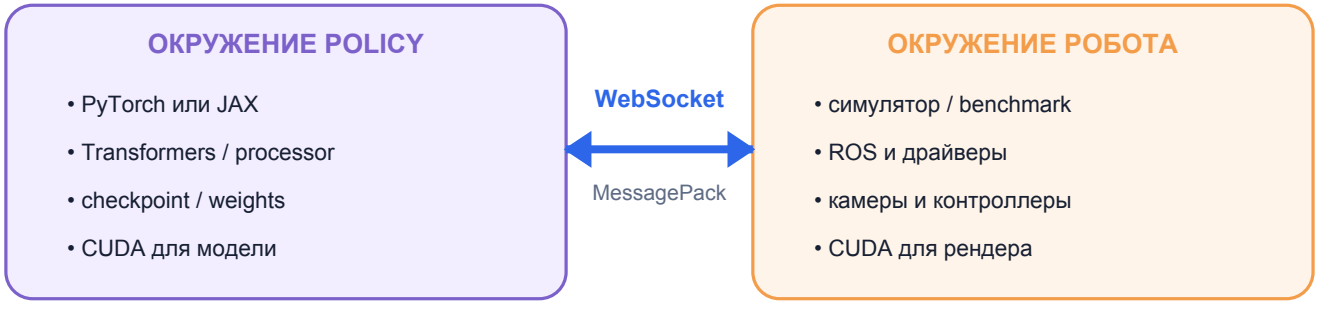
CUDA и GPU

GPU — физический процессор. CUDA — программная платформа NVIDIA, через которую фреймворк запускает kernels на GPU. Ещё есть driver, toolkit и собранные версии библиотек. Сообщение «CUDA mismatch» часто означает несовместимую комбинацию этих слоёв, а не неисправный GPU.

Зачем dependency isolation

Model A может требовать JAX и определённый CUDA stack, Model B — другую версию PyTorch, а simulator — собственную версию Python и рендера. Если установить всё в одно окружение, конфликтуют torch, numpy, transformers, CUDA extensions и системные пакеты.

Разные процессы, разные зависимости, при желании разные компьютеры



ИНФРАСТРУКТУРА XPOLICYLAB

XPolicyLab решает конфликт архитектурно: policy server и environment client — разные процессы. Они делятся только сериализованными данными, а не Python-объектами и импортами.

GPU separation

Аргументы policy_gpu_id и env_gpu_id позволяют запускать модель, например, на GPU 0, а simulator render/physics — на GPU 1. Это снижает конкуренцию за VRAM и вычисления. На машине с одной GPU оба id могут быть 0, но риск нехватки памяти выше.

11. Что где живёт в репозитории

Ниже — структура на commit 092782d. Репозиторий быстро развивается, поэтому список policy может увеличиться.

```
XPolicyLab/
├── README.md                # концепции, форматы, quick start
├── XPolicyLab.py            # модуль/путь совместимости пакета
├── pyproject.toml           # базовые зависимости Python >= 3.10
├── model_template.py        # минимальный Model API
├── setup_policy_server.py   # загрузка Model и запуск ws/legacy tcp
├── debug_env_client.py      # offline smoke check без simulator
├── client_server/
│   ├── ws/                  # WebSocket protocol, client, server, codec
│   └── tcp/                 # legacy transport
├── policy/
│   ├── demo_policy/         # минимальный эталон
│   ├── Pi_0/, Pi_05/        # OpenPI adapters
│   ├── OpenVLA_OFT/         # OpenVLA-OFT adapter
│   ├── GR00T_N17/           # NVIDIA adapter
│   ├── ACT/, DP/            # imitation baselines
│   └── ...                  # остальные adapters
├── scripts/                 # scaffold, downloads, converters
├── utils/                   # data, checkpoints, ports, launch helpers
└── assets/                  # схемы README
```

client_server/ws

Файл	Назначение
model_server.py	Держит Model, dispatch методов, lock, response cache, restart identity
model_client.py	Синхронная оболочка для environment; отдельный asyncio loop для keepalive
protocol/codec.py	MessagePack + NumPy, tensor→NumPy, проверки dtype и map keys
protocol/messages.py	Типы HELLO, RESET, CALL, INFER, TRIAL_END, HEARTBEAT, ERROR
protocol/schemas.py	Pydantic-схема Frame/envelope и поля идентификации
protocol/exceptions.py	Коды протокольных ошибок

policy/

Файл	Роль
README.md	Источник истины по установке, данным, checkpoint и ограничениям конкретной policy
install.sh	Создаёт model runtime и ставит зависимости
process_data.sh	Опционально переводит общий dataset в родной формат модели
train.sh	Опционально запускает обучение/fine-tuning
eval.sh	Same-machine orchestration: server + client + cleanup
setup_eval_policy_server.sh	Отдельный запуск model side
setup_eval_env_client.sh	Отдельный запуск environment side
deploy.yml	Runtime defaults и model-specific config
deploy.py	Evaluation loop: obs → remote calls → action
model.py	Adapter конкретной модели

Базовые зависимости пакета

В pyproject.toml: Python ≥3.10, NumPy, PyYAML, OpenCV headless, h5py, websockets, msgpack, msgpack-numpy и Pydantic. PyTorch/JAX не навязываются базовому пакету — тяжёлый стек ставит конкретный policy adapter.

12. Lifecycle: install → data → train → serve → evaluate

Фаза	Вход	Выход	Кто отвечает
Install	Исходники + manifests	Рабочий runtime	Policy adapter
Process data	Raw demos	TFDS / LeRobot / Zarr / HDF5…	Policy-specific converter
Train	Dataset + base model + config + seed	Checkpoint	Upstream trainer
Serve	Code + processor + checkpoint	Policy server на host:port	XPolicyLab + adapter
Evaluate	Task + environment + policy endpoint	Success/failure, logs, videos, metrics	Benchmark/client

Data processing

Разные модели ожидают разные datasets. π0/π0.5 используют LeRobot-путь, OpenVLA-OFT — ALOHA-format TFDS, Diffusion Policy — Zarr. XPolicyLab унифицирует верхнеуровневые скрипты и исходную schema, но реальное преобразование остаётся model-specific.

Training

Training обновляет weights по данным. Fine-tuning начинает с base checkpoint; training from scratch — с инициализации. В обоих случаях нужны согласованные image preprocessing, action normalization, robot embodiment и action space. Иначе inference-код формально загрузится, но выдаст бессмысленные команды.

Inference

Inference не меняет веса: загруженная модель вычисляет actions. Но stateful policy меняет временное состояние между шагами. Поэтому «inference deterministic» и «stateless» — разные свойства.

Evaluation

Benchmark определяет задачи, начальные состояния, лимиты времени и метрику успеха. Seed помогает воспроизвести случайную сцену, но полной битовой воспроизводимости могут мешать GPU kernels, физика и асинхронность.

Checkpoint naming

```
<bench_name>-<ckpt_name>-<env_cfg_type>-<action_type>-<seed>

Пример:
RoboDojo-cotrain-arx_x5-joint-0
```

Это соглашение помогает resolver-у найти run. Но конкретная policy может использовать upstream-native layout или явный path в deploy.yml.

13. Что нужно установить

Минимум для изучения и debug

- Git и Python 3.10+.
- Базовый пакет XPolicyLab через pip install -e ..
- Runtime выбранной policy — только если вы хотите запускать настоящую модель.
- Simulator/benchmark environment — только для sim evaluation; для debug не нужен.

Для настоящей VLA

- Достаточно VRAM/RAM и диска для weights.
- Совместимый NVIDIA driver/CUDA stack, если модель требует NVIDIA GPU.
- Processor/tokenizer/base checkpoint и fine-tuned checkpoint в ожидаемых путях.
- Доступ к model hub может требовать сеть, token или принятие лицензии.

Для симулятора/робота

- Отдельное environment runtime с самим simulator или benchmark.
- Для реального робота — ROS при необходимости, vendor SDK, drivers, network interface и безопасный controller.
- Калибровка камер, robot description, limits и action semantics.

СИМУЛЯТОР / РОБОТ

ROS и drivers не становятся частью VLA. Они находятся справа от network boundary и превращают стандартный action в реальные низкоуровневые команды.

Проверочный список совместимости

Проверка	Вопрос
Python	Совпадает ли версия с upstream policy?
GPU	Хватает ли VRAM отдельно модели и simulator?
CUDA	Совместимы ли driver, framework wheels и custom extensions?
Data	Камеры, color order, resolution, action normalization совпадают с training?
Robot	env_cfg_type, DoF, joint ordering, gripper и units совпадают?
Network	Доступны ли IP/port, не блокирует ли firewall?

14. Как запустить demo_policy

demo_policy — безопасный smoke test инфраструктуры. Это не обученная policy: она возвращает нулевые действия правильных ключей и размеров, обычно chunk длиной 2.

Шаг 1. Клонировать и поставить базовый пакет

```
mkdir demo_env
cd demo_env
git clone https://github.com/XPolicyLab/XPolicyLab.git
cd XPolicyLab
pip install -e .
```

Шаг 2. Установить demo adapter

```
cd policy/demo_policy
bash install.sh
# затем активируйте созданное policy environment, как указано скриптом
```

Шаг 3. Offline debug без simulator

```
export EVAL_ENV_TYPE=debug
bash eval.sh RoboDojo stack_bowls demo arx_x5 joint \
+  0 0 0 base base
```

Два последних base — placeholder names, которые demo adapter допускает для быстрой проверки. В реальном adapter передаются policy environment/uv path и environment conda env.

Что именно проверяется

- Импорт model.py и запуск policy server.
- Подключение client, HELLO, reset и последовательность RPC.
- Сериализация NumPy observation и server-side image decoding.
- Action keys и dimensions для выбранного robot config/action type.
- Batch logic.

Дополнительная проверка encoded images

```
DEBUG_OBS_ENCODED=1 EVAL_ENV_TYPE=debug bash eval.sh \
+  RoboDojo stack_bowls demo arx_x5 joint 0 0 0 base base
```

Debug client отправит три варианта camera color — JPEG buffer, raw bytes и plain array — чтобы проверить decode path.

ВАЖНО

Нулевое action не значит «всегда безопасно»: значение 0 может быть абсолютной целевой позицией, а не командой «не двигаться». demo_policy предназначена для IO/shape smoke check, а не для запуска без проверки на физическом роботе.

15. Как подключить свою модель

Практический маршрут

Шаг	Действие	Готово, когда…
1	Создать scaffold: scripts/create_policy.sh MY_POLICY	Есть policy/MY_POLICY с шаблонами
2	Описать upstream runtime и checkpoint в README/install.sh	Чистая машина может установить модель
3	Заполнить deploy.yml	Policy name, protocol, paths и defaults понятны
4	Реализовать Model.__init__	Загружаются именно нужные weights/device
5	Реализовать update_obs	Камеры/state/instruction превращаются в native input
6	Реализовать get_action	Результат — list[action_dict] с точными keys/shapes
7	Реализовать reset/batch	Нет утечки state между env/trials
8	Прогнать static и debug checks	IO проходит без simulator
9	Подключить simulator	Успешный короткий controlled trial

Решения, которые обязан принять adapter author

- Какие камеры обязательны, их порядок, crop/resize и normalization.
- Какой tokenizer/processor и prompt template использовать.
- Как построить proprioception vector и в каком порядке joints.
- Как денормализовать action, ограничить диапазон и выбрать chunk horizon.
- Что хранить между шагами и что очищать в reset.
- Как выровнять batch по env_idx_list.

Проверки из репозитория

```
git diff --check
bash -n policy/MY_POLICY/*.sh
python -m py_compile policy/MY_POLICY/model.py policy/MY_POLICY/deploy.py
# не декодировать XPolicyLab images вручную:
grep -rnE 'cv2\.imdecode|np\.frombuffer|Image\.open' policy/MY_POLICY/
```

Минимальный критерий качества

Adapter считается корректным не тогда, когда «модель что-то вернула», а когда input schema, preprocessing, checkpoint, action semantics, shape, units, limits, reset и batch поведение подтверждены тестами.

16. Как подключить свой simulator или robot

Environment client делает четыре вещи

- Получает observation от simulator/robot stack.
- Упаковывает его в XPolicyLab schema и отправляет policy server.
- Получает action chunk, валидирует и переводит в native commands.
- Шагает среду, считает termination/success и сообщает trial end.

Граница ответственности

Environment side	Policy side
Camera capture, calibration, timestamps	Image preprocessing для конкретной модели
Robot state и joint ordering source	Сбор native proprio vector
Action execution, IK/controller, limits	Prediction и model-side postprocessing
Physics, collisions, success metric	Weights, memory, chunk generation
ROS/vendor drivers	PyTorch/JAX/model dependencies

Same-machine и split-machine

На одной машине eval.sh поднимает обе стороны и чистит процессы. Для двух машин сначала запускают policy server с host 0.0.0.0, затем client подключают к реальному IP policy-машины. 0.0.0.0 — адрес прослушивания, не адрес назначения.

```
# policy machine
bash setup_eval_policy_server.sh ... <port> 0.0.0.0

# environment machine
bash setup_eval_env_client.sh ... <port> <POLICY_MACHINE_IP>
```

Безопасность реального робота

- Проверять NaN/Inf, shape, timestamp, units и допустимые диапазоны каждого action.
- Ограничивать скорость, ускорение, workspace и collision zones независимо от модели.
- Иметь watchdog: при потере связи отправлять безопасную команду и останавливать trial.
- Не считать WebSocket control loop hard real-time. Низкоуровневый стабилизирующий controller должен жить возле робота.
- Логировать observation/action/time для воспроизводимой диагностики.

17. Что XPolicyLab НЕ делает

Заблуждение	На самом деле
«Это VLA-модель»	Нет. Это инфраструктура; VLA — одна из категорий подключаемых policy.
«Он обучит любую модель одной командой»	Скрипты унифицированы по форме, но training code/data/checkpoints остаются model-specific.
«Любая policy работает на любом роботе»	Нужны совместимые cameras, embodiment, state/action semantics и адаптация.
«WebSocket управляет моторами напрямую»	Client передаёт action в simulator/robot stack; контроллеры и drivers остаются на стороне среды.
«Стандарт schema гарантирует качество»	Schema предотвращает часть интеграционных ошибок, но не делает weights хорошими.
«Checkpoint — это вся модель»	Нужны совместимые code, config, processor, normalization stats и runtime.
«Debug success значит robot success»	Debug проверяет wiring/shapes, не физический навык и не безопасность.
«Retry всегда безопасен»	Timeout может означать, что stateful call всё ещё выполняется; trial надо прервать.
«Seed делает всё полностью детерминированным»	Помогает воспроизводимости, но GPU/physics/asynchrony могут вносить различия.

Главное ограничение унификации

Общий интерфейс унифицирует форму общения, но не смысл автоматически. Две policy могут вернуть одинаковый shape и всё же использовать разные units, normalization, control frequency или joint order. Поэтому adapter и environment integration — инженерные контракты, а не просто переименование функций.

18. Диагностика: где искать проблему

Симптом	Вероятный слой	Первое действие
ImportError / missing package	Policy runtime	Проверить активное env и policy README/install
CUDA OOM	GPU allocation	Уменьшить batch/chunk/images; развести GPU; проверить VRAM
Client не подключается	Network/cold start	Проверить server log, host/port, firewall, retry budget
Connection drops	Keepalive/blocking/network	Проверить ping settings и server/client logs
ServerRestartedError	Policy server перезапущен	Прервать trial и заново reset/start
Timeout	Долгий/hung inference	Не повторять слепо; считать trial failed, смотреть server traceback
Неверный цвет	Image pipeline	Проверить RGB/BGR и запрет ручного decode
Action shape error	Adapter/action schema	Сверить env_cfg_type, DoF, arm/gripper keys
Движение странное, shape верный	Semantics	Проверить normalization, units, delta/absolute, joint order
Первый эпизод нормальный, второй нет	Stateful reset	Очистить history/hidden state/chunk caches
Batch env перепутаны	Batch bookkeeping	Сверить env_idx_list и per-env state

Где смотреть traceback

Клиент получает строковое сообщение ошибки, а полный traceback model failure логируется на **policy server side**. Поэтому при проблемах с inference сначала открывайте log процесса модели, а не только simulator terminal.

Порядок изоляции причины

```
1) импорт model.py
2) загрузка checkpoint без сети
3) один synthetic obs → update_obs/get_action
4) EVAL_ENV_TYPE=debug
5) debug with encoded images
6) simulator с одним env и малым horizon
7) batch / remote machine
8) только затем реальный robot
```

19. FAQ

XPolicyLab — это аналог ROS?

Скорее частичная аналогия. ROS — широкое middleware для компонентов робота. XPolicyLab сфокусирован на границе learned policy ↔ evaluation environment и стандартизирует adapter/lifecycle/serving.

Нужен ли ROS для XPolicyLab?

Нет для debug и многих simulator integrations. Для конкретного физического робота ROS может быть частью native env stack.

Можно ли запускать модель и simulator на разных компьютерах?

Да. Policy server слушает доступный interface/port, environment client подключается к IP policy machine. Нужно учесть firewall, latency и безопасность сети.

Нужны ли две GPU?

Не обязательно. Но раздельные GPU уменьшают конкуренцию за VRAM/compute. Для лёгкой policy или debug GPU вообще может не понадобиться.

Почему model.py не декодирует изображения?

Runtime server делает это до update_obs, чтобы contract был единым. Повторный decode ломает данные. Offline trajectory bits декодируются специальным helper.

Почему get_action возвращает список словарей?

Потому что это chunk: каждый элемент — action одного будущего control step, а словарь разделяет arm/gripper и левую/правую стороны.

Можно ли вернуть tensor?

Codec умеет преобразовать torch.Tensor в NumPy, включая CUDA→CPU, но лучше явно контролировать dtype/shape в adapter и не полагаться на неявное поведение.

Чем config отличается от checkpoint?

Config говорит, что и как собрать; checkpoint содержит обученное состояние. Неподходящая пара может не загрузиться или, хуже, загрузиться с неверной семантикой.

Что значит benchmark-agnostic?

Архитектура не привязана только к RoboDojo/RoboTwin. Но новый benchmark всё равно требует environment client и согласованную schema.

Как понять, VLA ли конкретная модель?

Смотрите paper/README: использует ли она vision и language для генерации robot actions. Наличие transformer или diffusion само по себе VLA не доказывает.

Можно ли использовать planner вместо нейросети?

Да, если обернуть его тем же Model interface и вернуть корректный action chunk.

20. Шпаргалка на одну страницу

ВСЕ POLICY

Policy: observation → action. VLA — лишь один класс policy. Action без точного space/units/order не определён.

ТОЛЬКО / ОСОБЕННО VLA

VLA: vision + language (+ state) → robot actions. Tokenizer работает с языком; processor подготавливает мультимодальный input; checkpoint несёт выученные weights.

ИНФРАСТРУКТУРА XPOLICYLAB

XPolicyLab: adapter + policy server + environment client + schema + lifecycle. Default transport — WebSocket; payload — MessagePack + NumPy.

СИМУЛЯТОР / РОБОТ

Environment: simulator/benchmark/robot, ROS, drivers, cameras, physics, controllers и success metric. Это не внутренность VLA.

Пять функций, которые надо помнить

```
update_obs(obs)                 # принять один observation
get_action()                    # вернуть один action chunk
reset()                         # очистить state между trials
update_obs_batch(obs_list)      # принять batch
get_action_batch(env_idx_list)  # вернуть batch chunks
```

Самый безопасный путь запуска

```
install base → demo_policy → EVAL_ENV_TYPE=debug
→ real adapter in debug → simulator single-env
→ simulator batch/remote → controlled real robot
```

Красные флаги

- RGB/BGR перепутаны; изображение декодируется дважды.
- Checkpoint, config, processor или normalization stats не от одного run.
- Joint order, units, absolute/delta или frequency не совпадают.
- reset не очищает history/chunk/hidden state.
- Timeout повторяется как обычный retry.
- Новая policy сразу отправляется на реальный робот без limits/watchdog/e-stop.

21. Полный словарь терминов

Эта таблица задумана как место, куда можно возвращаться при чтении README и papers.

Термин	Совсем простыми словами	Где живёт
Policy	Правило/модель, которая по observation выбирает action.	Все policy
VLA	Vision-Language-Action: модель, связывающая изображение, текстовую команду и действие робота.	Класс policy
Imitation learning	Обучение по демонстрациям: модель повторяет пары «наблюдение → действие» эксперта.	Способ обучения
Diffusion Policy	Policy, которая начинает с шума и несколькими шагами превращает его в согласованную траекторию действий.	Класс policy
World model	Модель, прогнозирующая, как изменится мир; не обязана напрямую управлять мотором.	Модель мира
World-action model	Семейство, где моделирование динамики/будущего тесно связано с выбором действий. Определения у работ различаются.	Класс policy/модели
Observation / obs	Снимок того, что доступно системе сейчас: камеры, суставы, команда, служебные поля.	Среда → policy
Action	Команда управления: целевые суставы, приращения, поза инструмента, состояние захвата и т. п.	Policy → среда
Action space	Точный язык команд: какие числа, в каком порядке, единицах и диапазонах.	Контракт
Action chunk	Несколько будущих действий, возвращённых одним inference-вызовом.	Выход policy
Stateful policy	Policy с памятью между шагами; reset обязан очистить память между эпизодами.	Все policy
Batch inference	Обработка нескольких сред одним запуском модели ради скорости GPU.	Все policy
RGB	Цветное изображение с тремя каналами: красный, зелёный, синий.	Датчики/данные
Depth / RGB-D	Карта расстояний; RGB-D означает цвет плюс глубина.	Датчики/данные
Proprioception	Внутреннее «ощущение тела»: углы/скорости суставов, захват, поза базы.	Робот/среда
Joint state	Текущее состояние суставов: обычно углы, иногда скорости и усилия.	Робот/среда
Joint action	Команда в координатах суставов, например целевые углы семи сочленений.	Action space
End-effector / EEF	Рабочий конец руки: захват, присоска, инструмент.	Механика робота
EE pose	Положение и ориентация рабочего конца; в XPolicyLab: [x,y,z,qw,qx,qy,qz].	Action/state
DoF	Degree of Freedom, независимая степень свободы. Семисуставная рука обычно имеет 7 DoF.	Механика робота
Checkpoint	Сохранённое состояние обучения; обычно веса плюс сопутствующие состояния/статистика.	Модель
Weights	Числовые параметры нейросети — то, чему она научилась.	Модель
Config	Настройки запуска: пути, размеры, режимы, адреса, таймауты.	Модель/инфраструктура
Processor	Предобработка входов и иногда постобработка выходов: resize, нормализация, упаковка.	Часто VLA
Tokenizer	Преобразователь текста — а иногда и действий — в дискретные token id.	Особенно VLA

Термин	Совсем простыми словами	Где живёт
Runtime	Среда исполнения: Python, библиотеки, системные зависимости и драйверы.	Инфраструктура
PyTorch / JAX	Фреймворки тензорных вычислений и обучения нейросетей.	Модель
CUDA	Платформа NVIDIA для выполнения вычислений на GPU.	Вычисления
GPU	Параллельный процессор, ускоряющий нейросети и часто симуляцию/рендер.	Вычисления
Simulator	Программа, имитирующая физику, робота, камеры и сцену.	Не VLA
Benchmark	Набор задач, правил и метрик для сравнимой оценки.	Не VLA
Environment	То, с чем взаимодействует policy: симулятор, benchmark-обвязка или реальный робот.	Не VLA
ROS	Middleware для компонентов робота: сообщения, сервисы, топики, драйверы.	Робот-инфраструктура
Driver	Низкоуровневый перевод команд программы в протокол конкретного устройства.	Робот-инфраструктура
Engine	Исполняющее ядро системы; смысл зависит от контекста: physics/render/inference engine.	Контекстный термин
Adapter	Переводчик между единым контрактом XPolicyLab и API конкретной модели.	XPolicyLab
Policy server	Процесс, который держит модель и отвечает на удалённые вызовы.	XPolicyLab
Environment client	Клиент возле среды: отправляет obs, получает action, исполняет его.	XPolicyLab/среда
WebSocket	Постоянное двустороннее сетевое соединение.	Транспорт
RPC	Идея «вызвать функцию в другом процессе/на другой машине».	Протокол
MessagePack	Компактная бинарная упаковка словарей, списков, чисел и массивов для сети.	Сериализация
Serialization	Преобразование объектов в байты и обратное восстановление.	Сеть
Retry	Повтор после обрыва/временной недоступности.	Надёжность
Keepalive	Ping/pong, проверяющий, что соединение ещё живо.	Надёжность
Seed	Начальное число генератора случайности для повторяемости эксперимента.	Эксперимент

22. Источники и границы актуальности

Репозиторий был заново скачан и проверен 4 сентября 2026 года. Зафиксирован commit 092782d22702b1600ed3ec3d17b48418b2e42a77 от 3 сентября 2026 года. Команды и число интеграций могут измениться после этой даты.

Основной источник

- XPolicyLab repository и README: github.com/XPolicyLab/XPolicyLab
- XPolicyLab paper: arXiv:2608.09892
- Project website: xpolicylab.github.io

Модели и методы

- π0: A Vision-Language-Action Flow Model for General Robot Control
- π0.5: A Vision-Language-Action Model with Open-World Generalization
- OpenPI code: github.com/Physical-Intelligence/openpi
- OpenVLA: An Open-Source Vision-Language-Action Model
- OpenVLA code: github.com/openvla/openvla
- NVIDIA Isaac GR00T: developer.nvidia.com/isaac/gr00t
- ACT: Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware
- Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

Benchmarks

- RoboDojo: github.com/RoboDojo-Benchmark/RoboDojo
- RoboTwin: github.com/RoboTwin-Platform/RoboTwin

Ограничения этого учебника

Гайд объясняет XPolicyLab и его контракт, но не заменяет README каждой policy, лицензии моделей, инструкции конкретного robot vendor и safety assessment. Детали WAM, GR00T и быстро развивающихся моделей следует сверять с их текущими primary sources.

ВАЖНО

Официальная схема репозитория включена напрямую. Исходное вложение по пути /mnt/data/image.png отсутствовало в доступной файловой системе этой сессии; по содержанию описания использована совпадающая официальная assets/infra.png и дан покомпонентный разбор.

Конец гайда. Если после чтения вы можете отдельно показать пальцем на model, adapter, server, client и robot controller — главная цель достигнута.